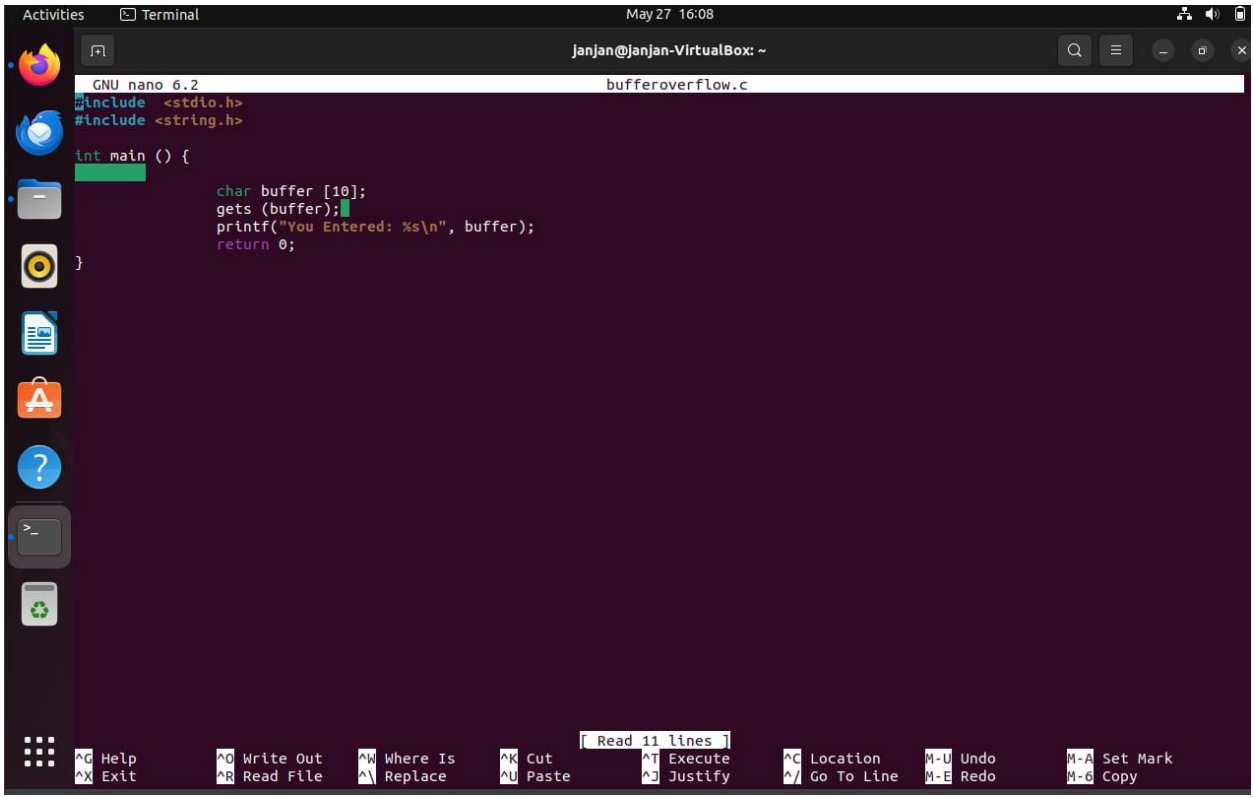
LISTANA, LANCE IVAN R. 05-26-25

BSIT-3B Sir. Benedick Sarmiento

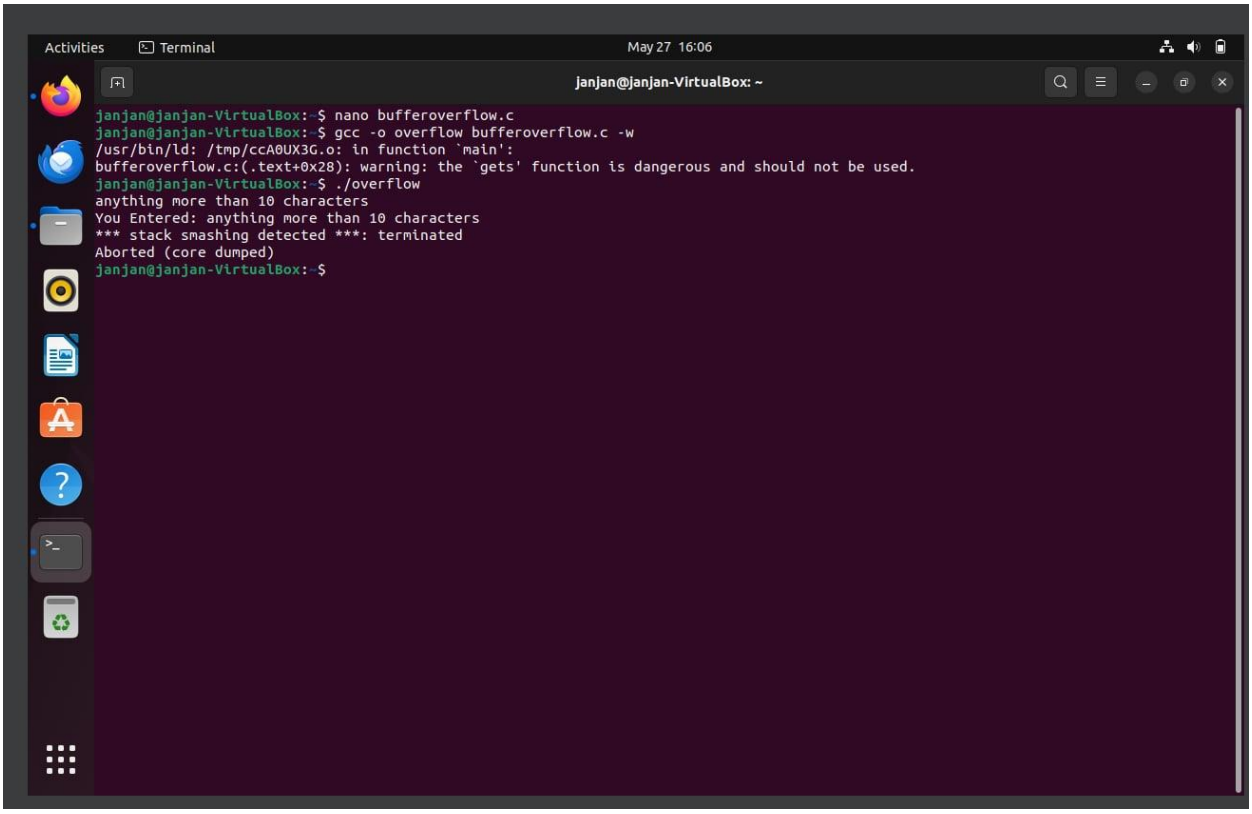
Common Security Vulnerabilities and Attacks

1. **Buffer Overflow (System-level)**

- **Entering The code to terminal**



- Executing The Code and Input System Buffer Overflows



Summary:

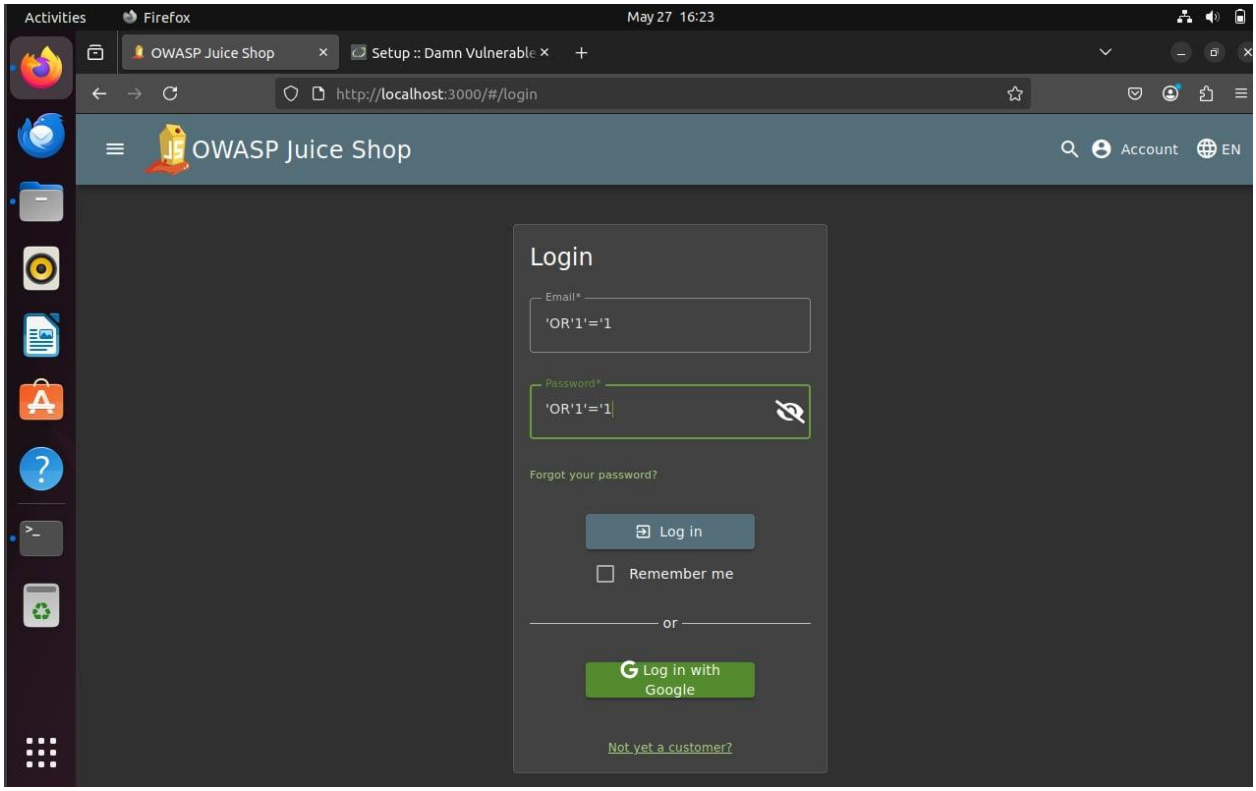
A buffer overflow happens when a program writes more data to a memory buffer than it can handle, potentially overwriting nearby memory and causing system instability or security issues. Fortunately, when I ran the code on my Ubuntu virtual machine, the system remained stable.

Question: How could modern compilers mitigate this?

Answer: By employing strategies like stack canaries, which identify memory corruption before it can impact program flow, modern compilers assist prevent buffer overflows. To make exploitation more challenging, they also offer features like address space layout randomization (ASLR) and bounds checking.

2. SQL INJECTION

- Injecting the SQL in login form (OWASP Juice Shop)



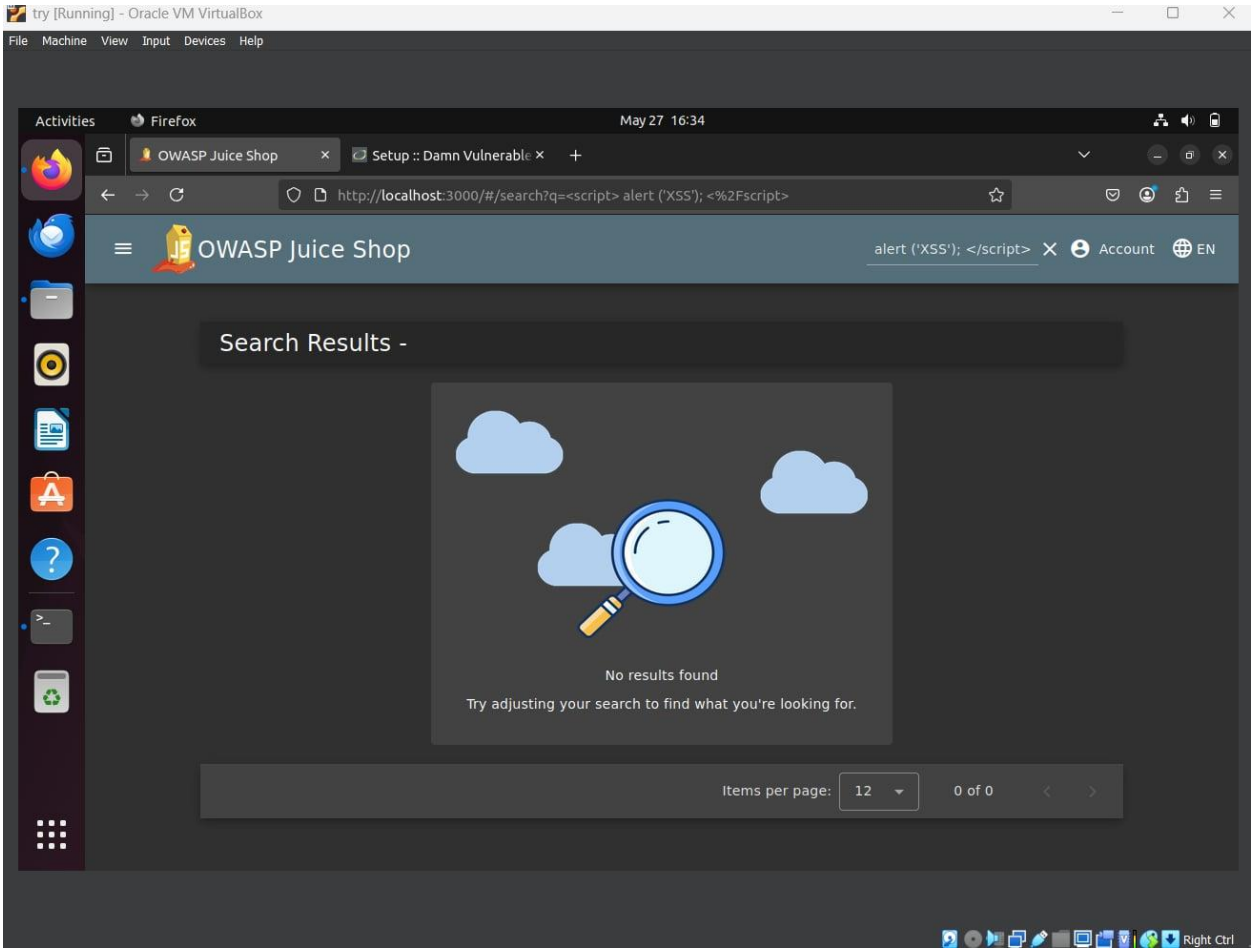
Summary: I was able to log in to the OWASP Juice Shop by entering ' OR '1' = '1', and even got an achievement for it. This worked because the app didn't properly filter user input, letting me change the SQL query and bypass the login.

Mitigation: Identify how parameterized queries prevent this**.**

Answer: Parameterized queries prevent SQL injection by keeping user input separate from the SQL code. Instead of inserting user input directly into the query, placeholders are used, and the input is treated as data, not code—so it can't change the structure of the SQL command.

3. **Cross-Site Scripting (XSS)**

- Executing alert script in OWASP Juice Shop search box



Note: This is the result when I used the script given in the instructions (It doesn't work)

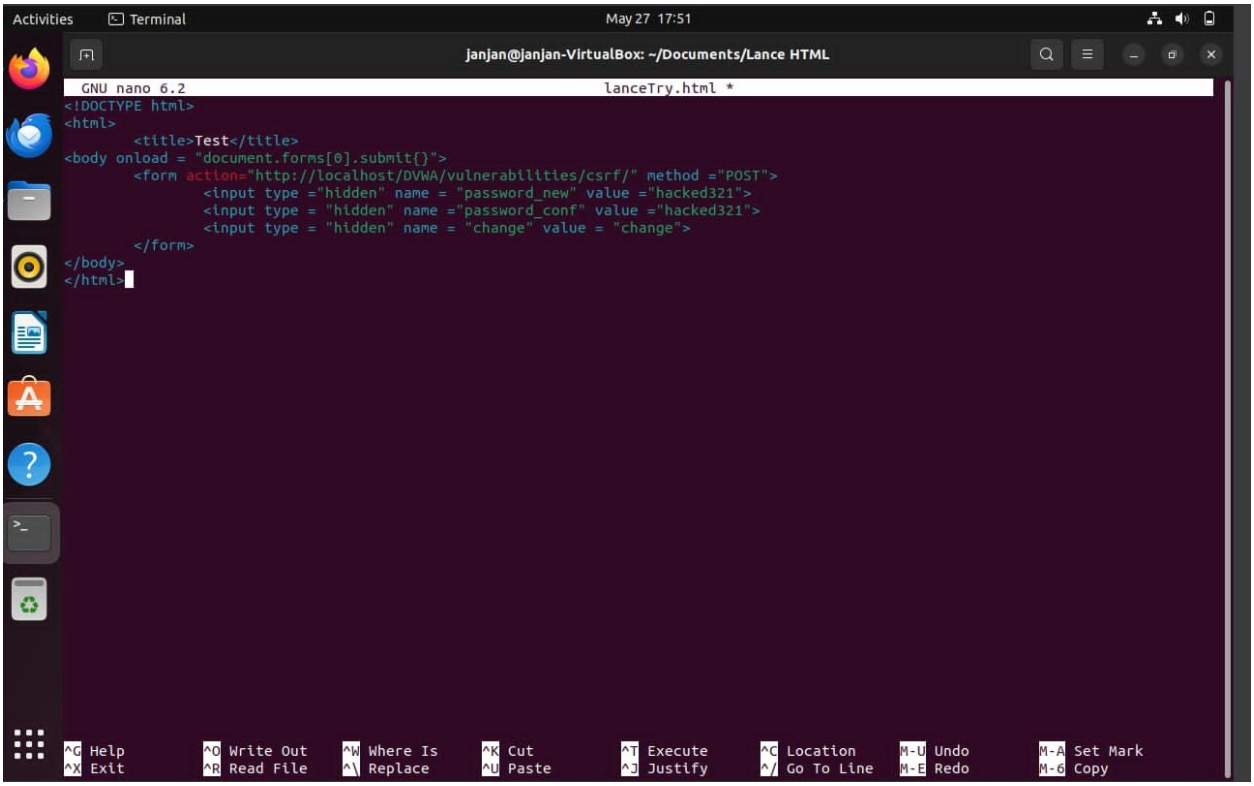
Summary: But the expected output is after injecting a script in the search box, an alert box popup in the website. This happens because the search box input isn't sanitized.

Reflection: Why is output encoding important?

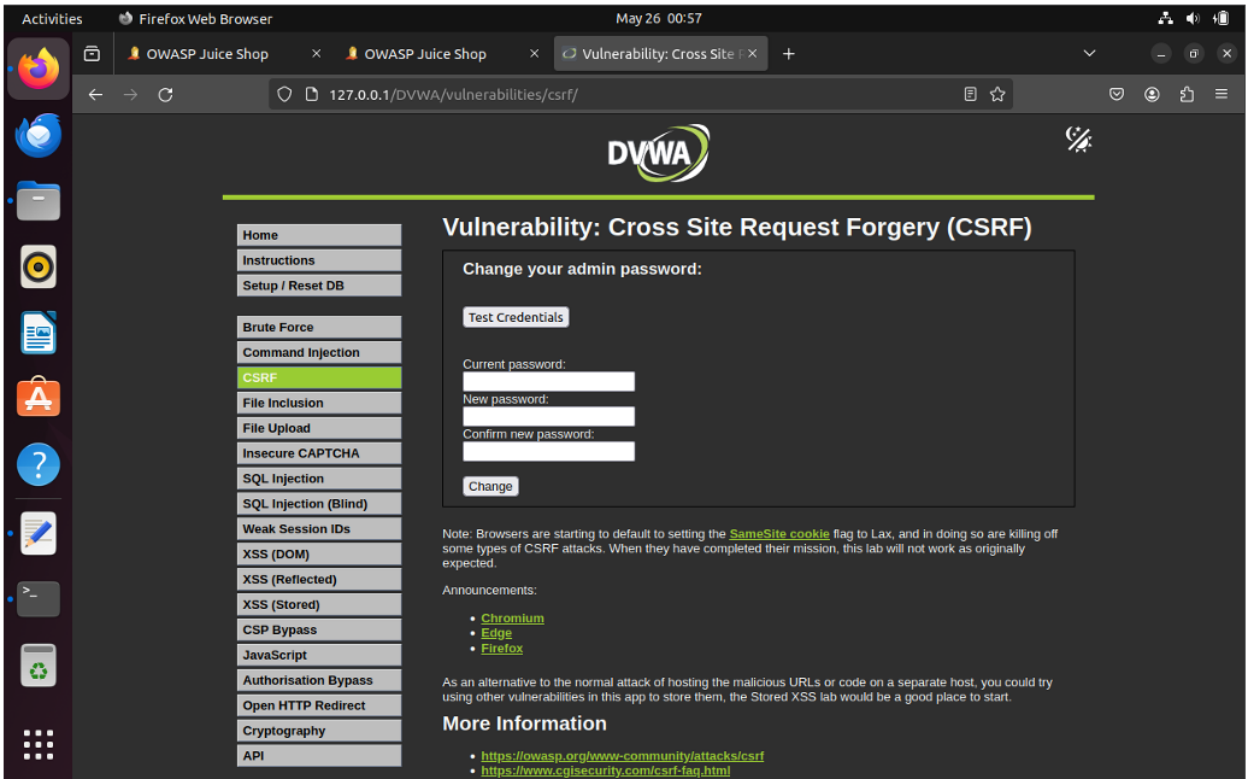
Answer: Output encoding is important because it helps stop hackers from putting harmful code into websites. It changes special characters into safe ones so the browser shows them as normal text instead of running them as code.

4. CSRF (Cross-Site Request Forgery)

- Creating the HTML File


Activities
Terminal
May 27 17:51
janjan@janjan-VirtualBox: ~/Documents/Lance HTML
GNU nano 6.2
lanceTry.html *
<!DOCTYPE html>
<html>
<title>Test</title>
<body onload = "document.forms[0].submit{}">
<form action="http://localhost/DVWA/vulnerabilities/csrf/" method ="POST">
<input type ="hidden" name = "password_new" value ="hacked321">
<input type = "hidden" name ="password_conf" value ="hacked321">
<input type = "hidden" name = "change" value = "change">
</form>
</body>
</html>
^G Help
^X Exit
^O Write Out
^R Read File
^W Where Is
^\ Replace
^K Cut
^U Paste
^T Execute
^J Justify
^C Location
^/ Go To Line
M-U Undo
M-E Redo
M-A Set Mark
M-6 Copy

- Form Forged


Activities
Firefox Web Browser
May 26 00:57
OWASP Juice Shop
OWASP Juice Shop
Vulnerability: Cross Site F
127.0.0.1/DVWA/vulnerabilities/csrf/
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
Vulnerability: Cross Site Request Forgery (CSRF)
Change your admin password:
Test Credentials
Current password:
New password:
Confirm new password:
Change
Note: Browsers are starting to default to setting the SameSite cookie flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.
Announcements:
Chromium
Edge
Firefox
As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.
More Information
https://owasp.org/www-community/attacks/csrf
https://www.cgisecurity.com/csrf-faq.html

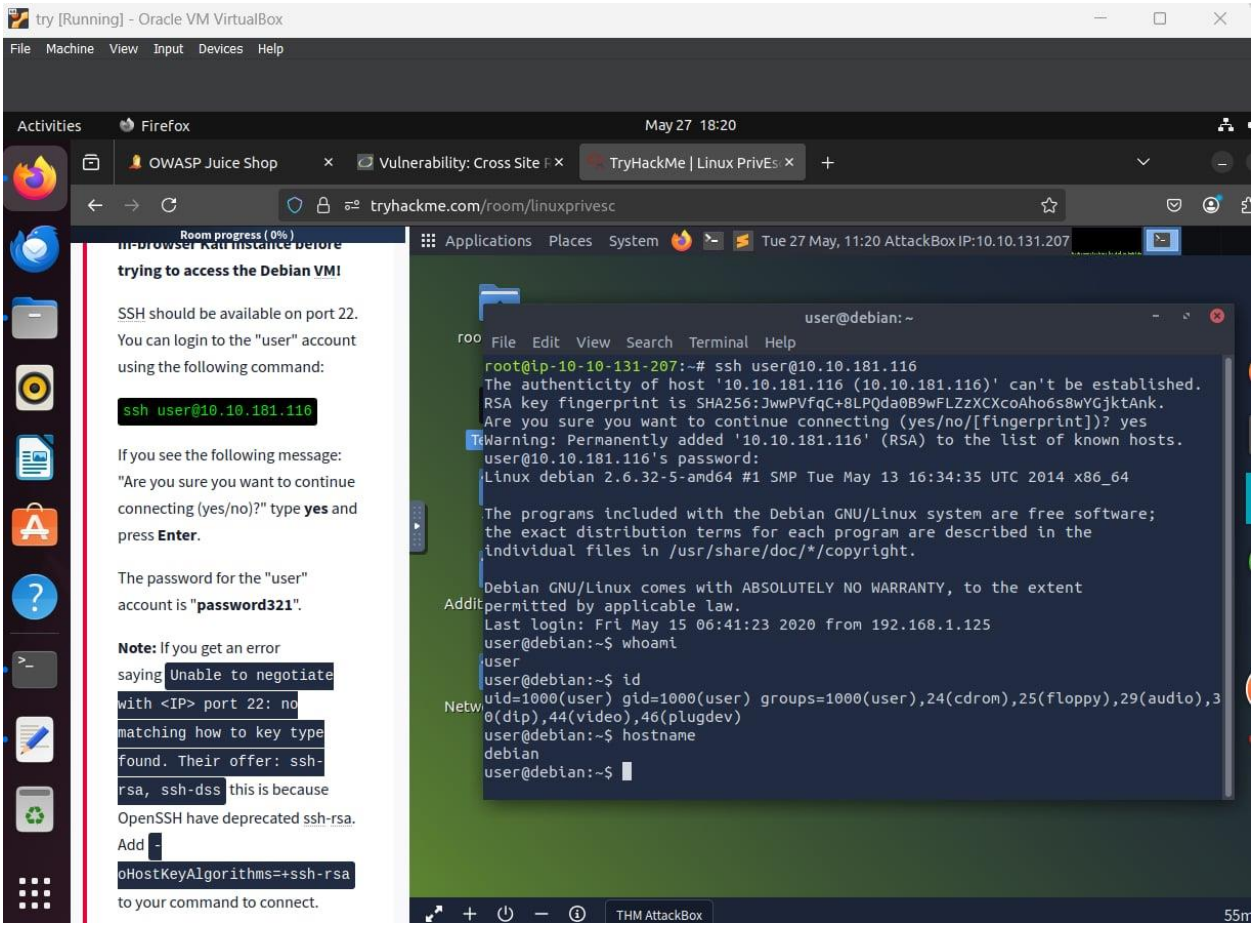
Summary:

At first, when I used the forged form to change my password, it didn't actually work, so I couldn't log in with the new password. But when I changed my password using the real form before trying any forgery, it worked correctly and I was able to log in. This shows that only the original, trusted form can properly update the password.

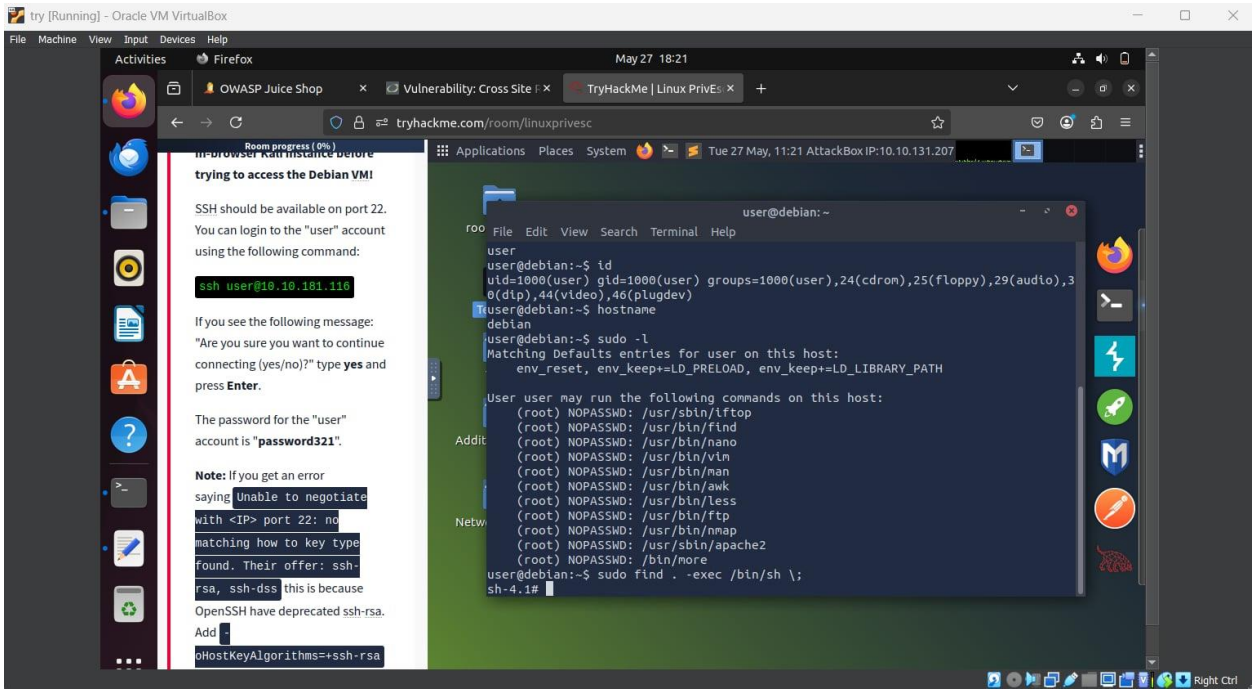
Discussion: How does using CSRF tokens stop this attack?
Answer:
CSRF tokens stop this kind of attack by making sure that any form submitted to the server comes from the real website, not from a forged one. The token is unique for each session and is checked by the server—if the token is missing or wrong, the request is blocked, preventing unauthorized actions like changing passwords.

5. Privilege Escalation

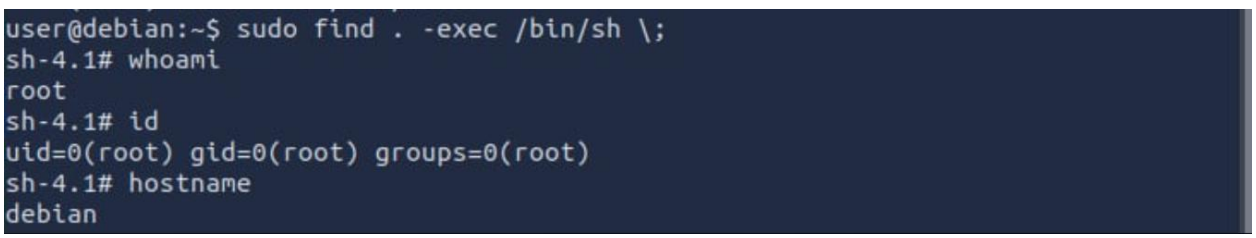
- Low-privileged user



- Checking sudo permissions and using ‘find’ to get to root access



- Root Access

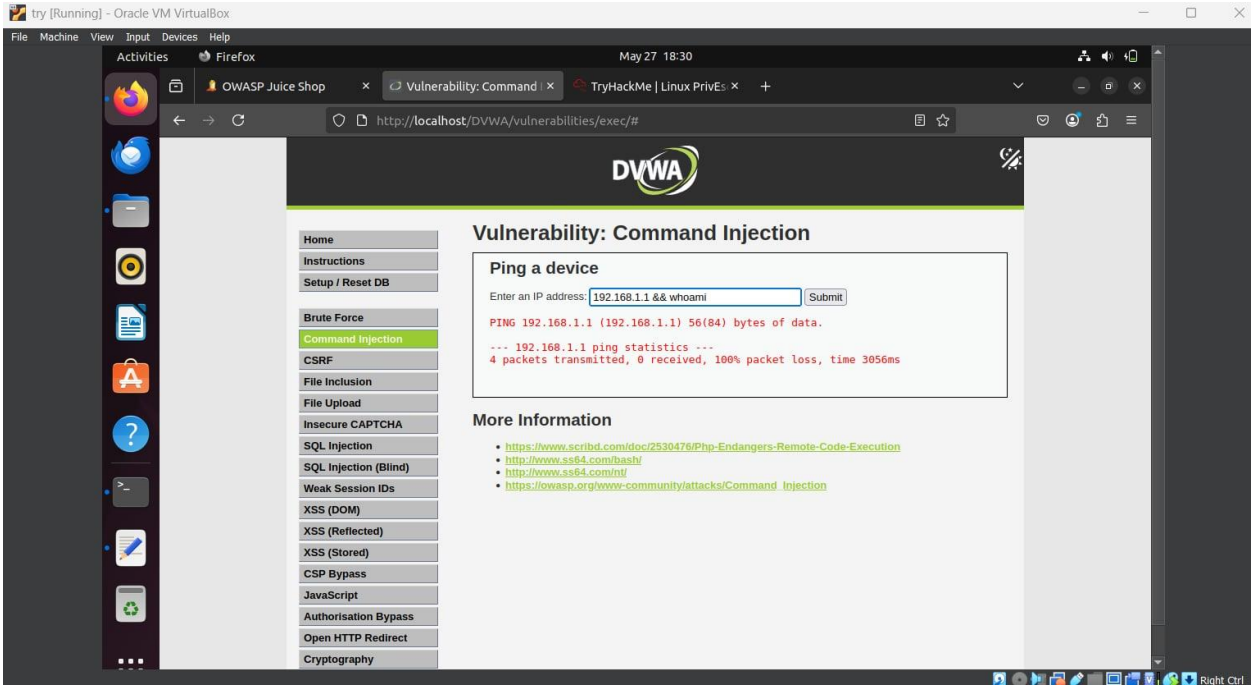


Summary:

I gained root access by taking advantage of a vulnerability in the system or a misconfiguration in its setup. This typically happens when security settings are weak, permissions are not properly managed, or known flaws in the software are left unpatched, allowing an attacker to escalate privileges and gain full control over the system.

6. Remote Code Execution (RCE)

- Executing remote commands through user input.



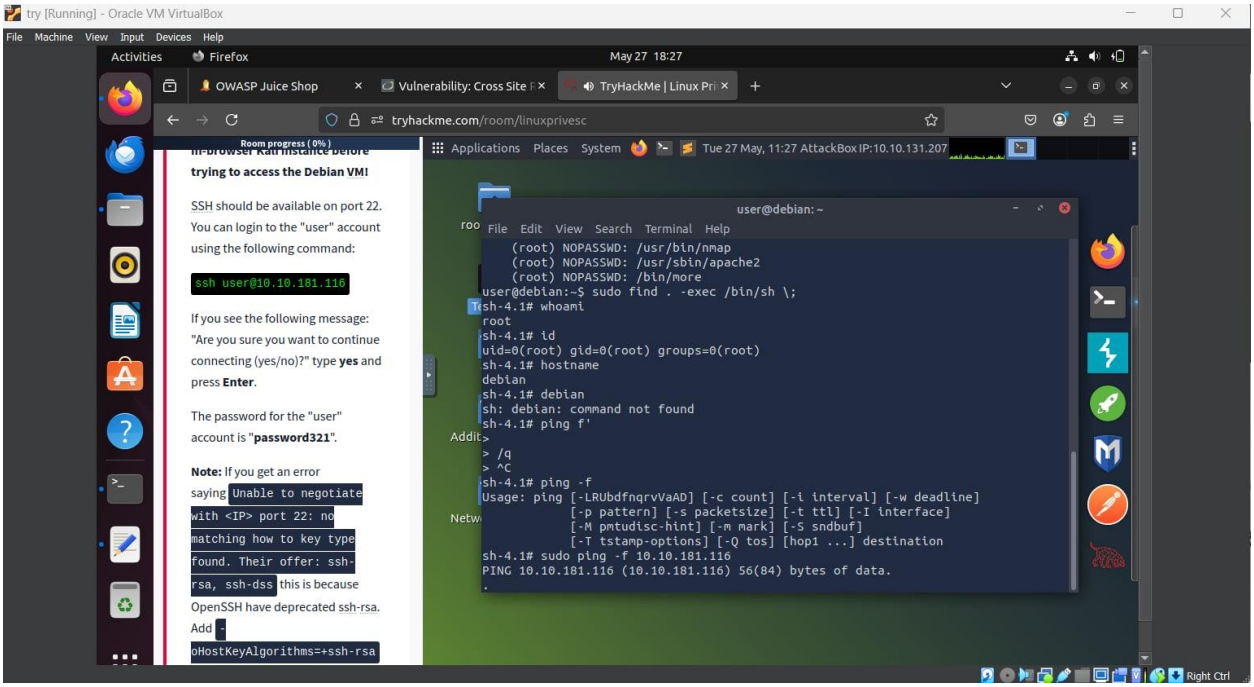
Summary: I used the input field in DVWA to ping an IP address, but by entering PING && whoami, I was also able to run another command that showed the current user. This worked because the input field doesn't block special characters like &&, which lets attackers add extra commands—this is known as command injection.

Defense: How can input sanitization stop this?

Answer: Input sanitization can stop this by detecting and removing or blocking special characters like && that are used to chain commands. By allowing only safe, expected input (like IP addresses in this case), it prevents attackers from injecting additional system commands.

7. Denial of Service (DoS)

- Executing of ping -f



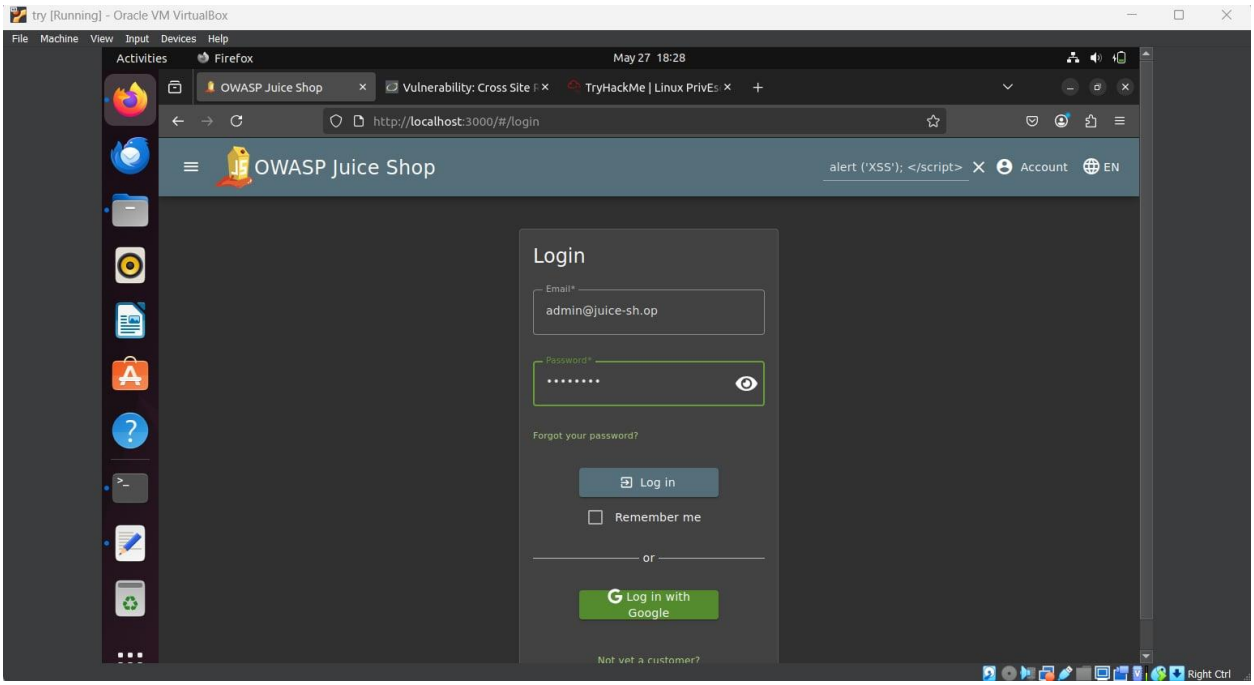
Summary: To safely test the dangerous ping -f command, I pinged my own IP address from TryHackMe. This caused a rapid stream of packets, simulating a DoS attack that can overwhelm a system and block access for real users.

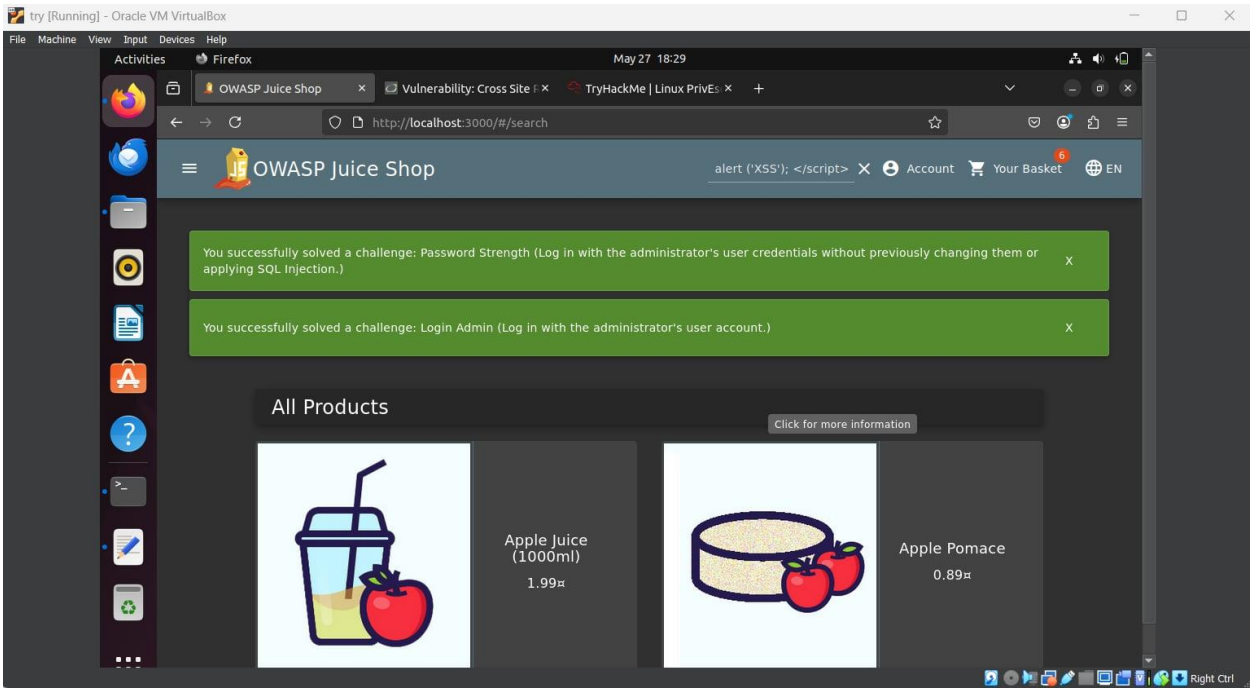
Discussion: What are rate limiting and firewalls' roles in defense?

Answer: Rate limiting helps defend against attacks by controlling how many requests a user or IP can send in a certain time period, preventing systems from being overwhelmed. Firewalls add another layer of defense by blocking suspicious or harmful traffic before it reaches the system, filtering out potential attacks like DoS or command injection.

8. **Authentic Bypass**

- Exploiting Default Credentials





Summary: I was able to access the website using default admin credentials, which worked because the system had weak security checks and didn't properly validate logins. This allowed me to bypass proper authentication and gain unauthorized access.

Mitigation: What practices help secure authentication mechanisms?

Answer: Using strong, unique passwords, turning on multi-factor authentication (MFA), and restricting login attempts to avoid brute-force attacks are all part of securing authentication

processes. In order to lower the risk of unwanted access, it also entails avoiding the usage of default or hardcoded credentials and safely storing passwords using salting and hashing.